

PL/SQL

1. Introduction to PL/SQL

PL/SQL (Procedural Language/Structured Query Language) is Oracle Corporation's procedural extension to SQL. While SQL is a non-procedural language used for data manipulation and definition, it lacks the ability to implement programming constructs such as decision-making, iteration, and modular design.

PL/SQL bridges this gap by integrating SQL with procedural constructs. It allows developers to write complete programs that can be stored and executed within the Oracle database.

Need for PL/SQL

- SQL executes one statement at a time and does not support control structures.
- Real-world applications require logic such as validation, looping, and error handling.
- PL/SQL enables embedding SQL statements inside procedural code.
- Reduces network traffic by executing multiple operations in a single block.

Key Idea

PL/SQL transforms SQL from a query language into a full-fledged programming environment within Oracle.

2. Features of PL/SQL

- **Block Structured Language:** Code is organized into logical blocks.
- **Tight Integration with SQL:** Direct use of SELECT, INSERT, UPDATE, DELETE.
- **Modularity:** Supports procedures, functions, and packages.
- **Improved Performance:** Entire block is sent to the database engine at once.
- **Error Handling:** Built-in exception handling mechanism.
- **Portability:** Works across all Oracle-supported platforms.

3. Structure of PL/SQL Block

PL/SQL programs are written in the form of blocks. Each block contains three sections.

General Syntax

```
DECLARE
  -- Variable declarations
BEGIN
  -- Executable statements
EXCEPTION
  -- Error handling statements
END;
/
```

Explanation of Sections

1. **DECLARE Section**
 - Optional
 - Used to declare variables, constants, cursors
2. **BEGIN Section**
 - Mandatory
 - Contains executable statements
 - SQL statements are written here
3. **EXCEPTION Section**
 - Optional
 - Handles runtime errors
4. **END**
 - Marks the end of the block

5. Slash (/)

- Required in SQL*Plus / Oracle XE to execute the block

4. Executing a Simple PL/SQL Program

Before executing any PL/SQL program, output must be enabled.

```
SET SERVEROUTPUT ON;
```

Example

```
BEGIN
  DBMS_OUTPUT.PUT_LINE('Welcome to PL/SQL');
END;
/
```

Explanation

- DBMS_OUTPUT.PUT_LINE is used to display output.
- Without enabling server output, no result will be visible.

5. Variables in PL/SQL

Variables are used to store values temporarily during program execution.

Syntax

```
variable_name datatype;
```

Rules

- Must be declared in DECLARE section
- Can store different types of values
- Assigned using := operator

Example

```
DECLARE
  v_name VARCHAR2(30);
  v_age NUMBER;
BEGIN
  v_name := 'Ravi';
  v_age := 20;

  DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);
  DBMS_OUTPUT.PUT_LINE('Age: ' || v_age);
END;
/
```

6. Data Types in PL/SQL

PL/SQL supports both SQL and additional procedural data types.

Common Data Types

Data Type	Description
NUMBER	Numeric values
VARCHAR2	Variable-length string
CHAR	Fixed-length string
DATE	Date and time
BOOLEAN	TRUE, FALSE, NULL

Using %TYPE Attribute

The %TYPE attribute allows a variable to inherit the datatype of a column.

Advantages

- Reduces errors
- Automatically adapts to table changes

Example

```
DECLARE
  v_salary emp.sal%TYPE;
BEGIN
  NULL;
END;
/
```

7. Input in PL/SQL

PL/SQL does not have built-in input functions like traditional programming languages.

Using Substitution Variables

```
DECLARE
  v_num NUMBER := &num;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Value entered: ' || v_num);
END;
/
```

Execution Behavior

- Prompts user for input
- Value replaces &num

8. Operators in PL/SQL

PL/SQL supports standard operators.

Types of Operators

1. Arithmetic: + - * /
2. Relational: =, <, >, <=, >=
3. Logical: AND, OR, NOT
4. String: || (concatenation)

9. Conditional Statements

Conditional statements allow decision-making in PL/SQL.

IF Statement

Syntax:

```
IF condition THEN
  statement1;
  statement2;
END IF;
```

```
DECLARE
  v_num NUMBER := 10;
BEGIN
  IF v_num > 5 THEN
    DBMS_OUTPUT.PUT_LINE('Number is greater than 5');
  END IF;
END;
/
```

IF-ELSE Statement

Syntax:

```
IF condition THEN
statements_if_true;
ELSE
statements_if_false;
END IF;
```

```
BEGIN
  IF 10 > 20 THEN
    DBMS_OUTPUT.PUT_LINE('Condition is true');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Condition is false');
  END IF;
END;
/
```

IF-ELSIF Ladder

Syntax:

```
IF condition1 THEN
  statements1;
ELSIF condition2 THEN
  statements2;
ELSIF condition3 THEN
  statements3;
ELSE
  default_statements;
END IF;
```

```
DECLARE
  v_marks NUMBER := 75;
BEGIN
  IF v_marks >= 80 THEN
    DBMS_OUTPUT.PUT_LINE('Distinction');
  ELSIF v_marks >= 60 THEN
    DBMS_OUTPUT.PUT_LINE('First Class');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Fail');
  END IF;
END;
/
```

10. Looping Statements

Loops are used to execute a block of code multiple times.

Basic LOOP

Syntax:

LOOP

statements;

EXIT WHEN condition;

END LOOP;

DECLARE

i NUMBER := 1;

BEGIN

LOOP

DBMS_OUTPUT.PUT_LINE(i);

i := i + 1;

EXIT WHEN i > 5;

END LOOP;

END;

/

WHILE LOOP

Syntax

WHILE condition LOOP

statements;

END LOOP;

DECLARE

i NUMBER := 1;

BEGIN

WHILE i <= 5 LOOP

DBMS_OUTPUT.PUT_LINE(i);

i := i + 1;

END LOOP;

END;

/

FOR LOOP

Syntax

FOR counter_variable IN lower_limit..upper_limit LOOP

statements;

END LOOP;

```

BEGIN
  FOR i IN 1..5 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
  END LOOP;
END;
/

```

Note

- Loop variable is automatically declared
- Cannot modify loop variable

REVERSE FOR LOOP

Syntax

```

FOR counter_variable IN REVERSE upper_limit..lower_limit LOOP
  statements;
END LOOP;

```

```

BEGIN
  FOR i IN REVERSE 5..1 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
  END LOOP;
END;
/

```

INTO Clause in PL/SQL

The INTO clause in PL/SQL is used to **store the result of a SQL query into variables**.

1. Definition : INTO is used with SELECT statement in PL/SQL to **fetch data from a table and assign it to variables**.

2. Basic Syntax

```

SELECT column1, column2, ...
INTO variable1, variable2, ...
FROM table_name
WHERE condition;

```

3. Important Rules

1. Number of columns must match number of variables
2. Datatypes must be compatible
3. Query must return **only one row**
4. Used only in PL/SQL (not in normal SQL)

Example: Fetch employee name and salary using EMPNO

```

DECLARE
  v_name EMP.ENAME%TYPE;
  v_sal EMP.SAL%TYPE;
BEGIN
  SELECT ENAME, SAL
  INTO v_name, v_sal
  FROM EMP
  WHERE EMPNO = 7369;

```

```
DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);
DBMS_OUTPUT.PUT_LINE('Salary: ' || v_sal);
END;
/
```

Using Expressions with INTO

```
DECLARE
    v_total NUMBER;
BEGIN
    SELECT SAL + NVL(COMM,0)
    INTO v_total
    FROM EMP
    WHERE EMPNO = 7499;

    DBMS_OUTPUT.PUT_LINE('Total Salary: ' || v_total);
END;
/
```

Using INTO with %ROWTYPE

Instead of multiple variables, we can store entire row.

```
DECLARE
    v_emp EMP%ROWTYPE;
BEGIN
    SELECT *
    INTO v_emp
    FROM EMP
    WHERE EMPNO = 7369;

    DBMS_OUTPUT.PUT_LINE(v_emp.ENAME || ' - ' || v_emp.SAL);
END;
/
```

9. Common Errors with INTO

1. NO_DATA_FOUND

Occurs when query returns no rows

-- EMPNO does not exist

2. TOO_MANY_ROWS

Occurs when query returns more than one row

-- Missing WHERE clause or multiple matching rows